CSA04 – STATISTICS WITH R PROGRAMMING

CO1 – ASSESSMENT TOOL 3 | ASSIGNMENT

P.Vignesh

192525013

Question 1

A college wants to develop a student record management system that stores student ID, name, department, and marks. Design an S3 class in R to represent student records and implement a generic function to display student details. Demonstrate object creation and explain how method dispatch is performed.

Answer

S3 is a simple and flexible object-oriented system in R. An S3 object is usually a list or another R object with a class attribute. A generic function calls a method based on the class of the object supplied to it.

R Program

```
# Create an S3 student record
student <- function(id, name, department, marks) {
  record <- list(
    student_id = id,
    name = name,
    department = department,
    marks = marks
  )
  class(record) <- "Student"
  return(record)
}

# Define a generic function
display_details <- function(x) {
  UseMethod("display_details")
}

# Define the Student-specific method
display_details.Student <- function(x) {
  cat("Student ID :", x$student_id, "\n")
  cat("Name       :", x$name, "\n")
  cat("Department :", x$department, "\n")
  cat("Marks      :", x$marks, "\n")
}
```

```
# Create and display an object
s1 <- student(101, "Anitha", "Computer Science", 92)
display_details(s1)
```

Sample Output

```
Student ID : 101
Name       : Anitha
Department : Computer Science
Marks      : 92
```

Question 2

A hospital requires a patient management system to maintain patient ID, age, and body temperature with predefined data types. Implement an S4 class in R with appropriate slots and create patient objects. Demonstrate slot access and explain how S4 classes help enforce data consistency.

Answer

S4 is a formal object-oriented system in R. A class is defined using setClass(), and its fields are declared as slots with specified data types. Objects are created using new(). This formal structure helps detect invalid slot types and missing required slots.

R Program

```
setClass(
  "Patient",
  slots = c(
    patient_id = "character",
    age = "numeric",
    temperature = "numeric"
  )
)

# Create a patient object
p1 <- new(
  "Patient",
  patient_id = "P101",
  age = 35,
  temperature = 37.2
)

# Access slots using @
cat("Patient ID :", p1@patient_id, "\n")
cat("Age        :", p1@age, "\n")
cat("Temperature:", p1@temperature, "C\n")
```

Sample Output

```
Patient ID : P101
Age        : 35
Temperature: 37.2 C
```

Question 3

A warehouse needs to manage product ID, product name, stock quantity, and unit price. Develop a Reference Class in R with methods to update stock quantity and calculate the total inventory value. Demonstrate how changes made to an object are reflected when the same object is accessed elsewhere.

Answer

Reference Classes in R are mutable: an object can be changed by calling one of its methods, and the change is visible through other variables referring to that same object. They are defined with setRefClass().

R Program

```
Product <- setRefClass(
  "Product",
  fields = list(
    product_id = "character",
    product_name = "character",
    stock_quantity = "numeric",
    unit_price = "numeric"
  ),
  methods = list(
    update_stock = function(amount) {
      new_quantity <- stock_quantity + amount
      if (new_quantity < 0) {
        stop("Stock quantity cannot be negative.")
      }
      stock_quantity <<- new_quantity
      invisible(stock_quantity)
    },
    inventory_value = function() {
      stock_quantity * unit_price
    }
  )
)

# Create a product object
item1 <- Product$new(
  product_id = "PR101",
  product_name = "Notebook",
  stock_quantity = 50,
  unit_price = 40
```

```
)

cat("Initial stock:", item1$stock_quantity, "\n")
cat("Initial inventory value:", item1$inventory_value(), "\n")

# Another variable refers to the same reference object
item2 <- item1
item2$update_stock(10)

cat("Stock through item1:", item1$stock_quantity, "\n")
cat("Stock through item2:", item2$stock_quantity, "\n")
cat("Updated inventory value:", item1$inventory_value(), "\n")
```

Sample Output

```
Initial stock: 50
Initial inventory value: 2000
Stock through item1: 60
Stock through item2: 60
Updated inventory value: 2400
```

Question 4

A banking application uses S3 classes to represent savings and current accounts, each requiring a different interest calculation method. Analyse how generic functions and method dispatch can be used to implement account-specific behaviour. Support your explanation with a suitable R program.

Answer

S3 generic functions allow the same function name to behave differently depending on an object's class. A savings account and a current account can share a generic such as calculate_interest(), while each class has its own method.

R Program

```
# Generic function
calculate_interest <- function(account) {
  UseMethod("calculate_interest")
}

# Constructors for two account types
savings_account <- function(balance, rate) {
  x <- list(balance = balance, rate = rate)
  class(x) <- "SavingsAccount"
  x
}

current_account <- function(balance, rate) {
  x <- list(balance = balance, rate = rate)
```

```
  class(x) <- "CurrentAccount"
  x
}

# Savings accounts earn interest at the given annual rate
calculate_interest.SavingsAccount <- function(account) {
  account$balance * account$rate
}

# Current accounts use a different policy: no interest here
calculate_interest.CurrentAccount <- function(account) {
  0
}

s1 <- savings_account(10000, 0.05)
c1 <- current_account(10000, 0.01)

cat("Savings account interest:", calculate_interest(s1), "\n")
cat("Current account interest:", calculate_interest(c1), "\n")
```

Sample Output

```
Savings account interest: 500
Current account interest: 0
```

Analysis of Dispatch

UseMethod("calculate_interest") checks the class of the supplied object. For a SavingsAccount object, R invokes calculate_interest.SavingsAccount(); for a CurrentAccount object, it invokes calculate_interest.CurrentAccount(). The current-account rule in this example is an illustrative policy that pays no interest; an actual bank would implement its own account terms. The approach keeps the calling code simple while allowing account-specific behaviour.

Question 5

An organization plans to develop an employee management system requiring predefined data types, validation of employee details, inheritance, and modification of shared object state. Compare S3, S4, and Reference Classes in R based on these requirements and justify the most suitable approach for the application.

Answer

S3, S4, and Reference Classes all support object-oriented programming in R, but they differ in formality, validation, inheritance, and how object changes are handled.

Requirement	S3	S4	Reference Classes
Predefined data types	Flexible list fields; types are not formally declared by the class	Slots declare required R classes/types	Fields can declare required R classes/types
Validation	Usually implemented manually in constructors/functions	Formal slot checks; custom validity rules can be added	Field types and custom method checks can be used

Inheritance	Supported through class vectors and method lookup; relatively informal	Explicit class inheritance can be defined	Supports reference-class inheritance
Shared mutable state	Usually copy-on-modify; not designed around shared mutation	Generally value-based; not designed around shared mutation	Mutable reference objects; changes are visible through aliases
Complexity	Simple and lightweight	More formal and structured	Useful for stateful objects and methods